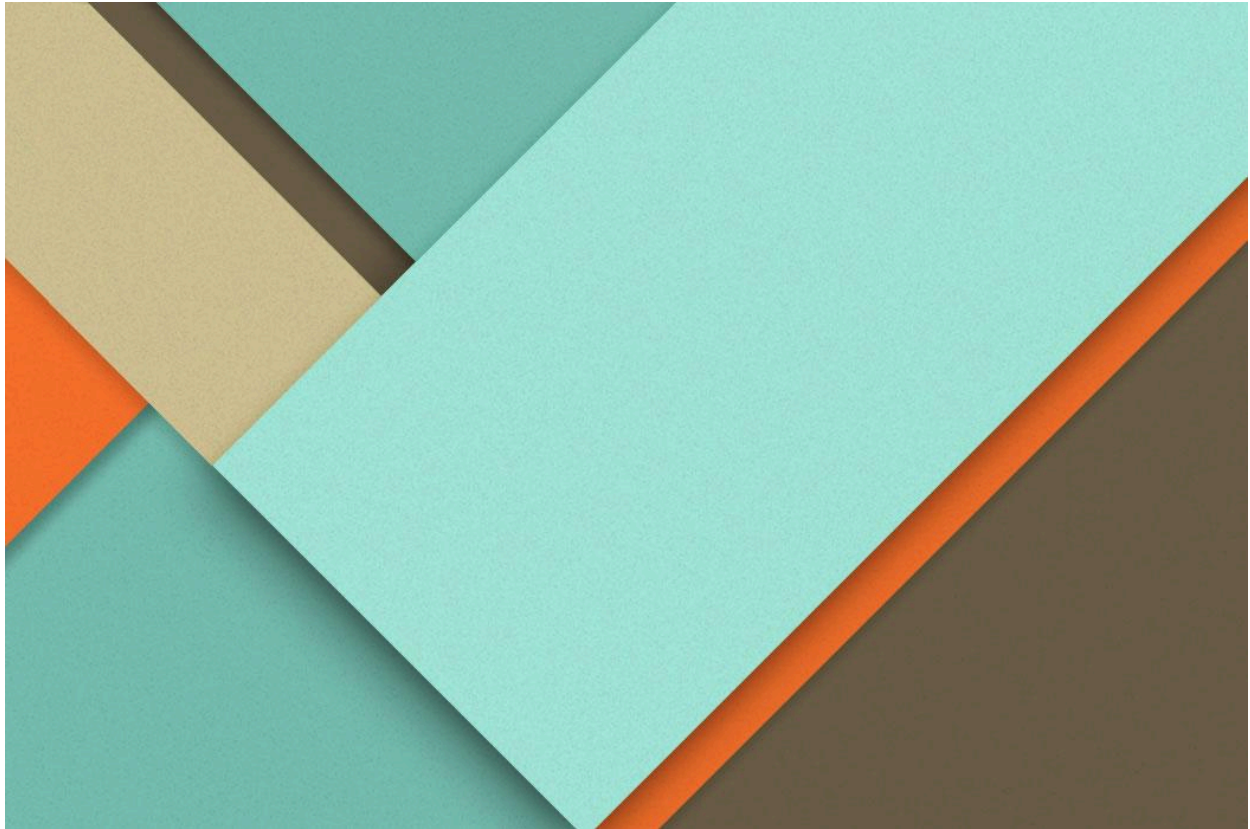




AF4: Tarea Dual AaD (XML)

22/03/2026

—

José Manuel Sánchez Rosal

2º DAM

IES Antonio Gala

1400 Palma del Río (Córdoba)

1 Índice

1 Índice.....	0
2 Análisis de uso del formato XML en el entorno laboral.....	1
Comparativa frente a otros formatos.....	1
3 Creación y manipulación de documentos XML.....	2
Operaciones CRUD implementadas (DOM).....	3
4 Gestión y organización de colecciones XML.....	4
5 Conclusiones.....	5

2 Análisis de uso del formato XML en el entorno laboral

A pesar del auge de formatos más ligeros como JSON, XML sigue siendo el estándar indiscutible en contextos críticos y empresariales. A continuación, se detallan documentos reales y sus contextos de uso:

- **Factura Electrónica (FacturaE):** Es el formato obligatorio para el intercambio tributario entre las empresas y la Administración Pública. Su uso permite integrar la Firma Digital (XML-DSig) y realizar una validación estricta de la estructura del documento.
- **Servicios Web SOAP:** Ampliamente utilizado en integraciones bancarias y grandes sistemas de reservas, como Amadeus o Sabre. XML proporciona contratos de interfaz estrictos mediante WSDL y altos niveles de seguridad con WS-Security.
- **Configuración de Entornos:** Archivos como [pom.xml](#) en proyectos Maven, [web.xml](#) en aplicaciones Java EE o el Android Manifest. Su principal ventaja aquí es la representación jerárquica clara de dependencias y parámetros complejos.

Comparativa frente a otros formatos

Para entender su valor, es fundamental compararlo con otras alternativas del mercado:

Característica	XML	JSON	CSV	SQL (Relacional)
Estructura	Árbol de etiquetas (Verboso)	Clave-Valor (Ligero)	Tabular (Plano)	Tablas relacionales
Validación	Nativa mediante XSD o DTD	Externa (JSON Schema)	Inexistente	Estricta (Esquemas, Claves foráneas)
Tipado	Todo es texto, salvo que se use XSD	Cadenas, Números, Booleanos	Texto plano	Tipos de datos estrictos
Uso Ideal	Documentos legales y configuración	APIs REST, Mobile y Web	Data Science y Excel	Transacciones complejas

3 Creación y manipulación de documentos XML

Para la parte práctica, se ha desarrollado una aplicación en Java que gestiona una pequeña base de datos simulada en el archivo `productos.xml`. Esta gestión se realiza íntegramente utilizando la API **Java DOM** (org.w3c.dom), la cual permite cargar el árbol de nodos en memoria, manipularlo y persistir los cambios mediante `javax.xml.transform`.

La estructura lógica del documento generado es la siguiente:

```
<tienda>
  <producto id="1">
    <nombre>Portátil Gaming</nombre>
    <descripcion>MSI 16GB RAM</descripcion>
    <precio>1200.50</precio>
    <stock>5</stock>
  </producto>
</tienda>
```

Operaciones CRUD implementadas (DOM)

A continuación, se documenta el proceso con los fragmentos clave del código desarrollado:

1. Lectura de Nodos (Read): El sistema carga el archivo físico y obtiene una `NodeList` con todos los elementos `<producto>` iterando sobre ellos para extraer el contenido de sus etiquetas hijas.

```
Document doc = cargarDOM(archivo);
NodeList lista = doc.getElementsByTagName("producto");
for (int i = 0; i < lista.getLength(); i++) {
    Element e = (Element) lista.item(i);
    System.out.printf("ID: %s | %s | %s€ | Stock: %s\n",
        e.getAttribute("id"),
        e.getElementsByTagName("nombre").item(0).getTextContent(),
        e.getElementsByTagName("precio").item(0).getTextContent(),
        e.getElementsByTagName("stock").item(0).getTextContent());
}
```

2. Inserción de Nodos (Create): Se crea un nuevo `Element` padre y se le asigna un atributo `id`. Posteriormente, se crean los elementos hijos (nombre, precio, etc.) y se añaden al nodo raíz (`<tienda>`).

```
Element producto = doc.createElement("producto");
producto.setAttribute("id", id);
crearElemento(doc, producto, "nombre", nom);
// ... creación de otros elementos hijos ...
root.appendChild(producto);
guardarXML(doc, archivo);
```

3. Modificación de Nodos (Update): Se busca el producto por su atributo `id`. Al encontrarlo, se localiza la etiqueta `<precio>` y se modifica su contenido textual con `setTextContent()`.

```
if (e.getAttribute("id").equals(idBuscado)) {
    e.getElementsByTagName("precio").item(0).setTextContent(nuevoPrecio);
    encontrado = true;
    break;
}
```

4. Eliminación de Nodos (Delete): Se localiza el nodo a eliminar iterando la lista. Una vez encontrado, se utiliza `getParentNode().removeChild(e)` para extraer el nodo del árbol DOM.

```
if (e.getAttribute("id").equals(idBuscado)) {
    e.getParentNode().removeChild(e);
    encontrado = true;
    break;
}
```

4 Gestión y organización de colecciones XML

El proyecto también incluye un módulo para la organización lógica de archivos en el sistema. A través del método `gestionarColecciones()`, la aplicación demuestra la capacidad de crear y eliminar estructuras de directorios organizadas por años, generando la ruta `coleccion_xml/2024/reporte.xml`.

Mantenimiento en un entorno de trabajo real: En un escenario corporativo, estas colecciones no se administrarían manualmente. Se emplearían procesos automatizados que generarían jerarquías lógicas estructuradas por año, mes y departamento (por ejemplo: `/facturas/2024/10/Ventas/`).

Para asegurar un mantenimiento eficiente a largo plazo, se deben implementar las siguientes estrategias:

- **Políticas de retención (Hot vs. Cold Storage):** Los archivos XML recientes se mantendrían en sistemas de almacenamiento rápido para acceso inmediato. Por el contrario, los documentos de ejercicios anteriores se comprimirían y migrarían a servidores de respaldo más económicos (Cold Storage).
- **Políticas de inmutabilidad:** Se aplicarían permisos estrictos de solo lectura a los directorios que contienen datos históricos para prevenir modificaciones accidentales o malintencionadas, garantizando la integridad legal de la información.

5 Conclusiones

El desarrollo de este proyecto ha permitido consolidar los conocimientos exigidos en el Resultado de Aprendizaje 5 (RA5), demostrando la capacidad práctica para gestionar bases de datos nativas XML de forma efectiva. De la implementación técnica y el análisis teórico realizado se extraen las siguientes conclusiones:

- **Eficacia de la API DOM:** La utilización de las clases `org.w3c.dom` en Java ha demostrado ser una herramienta robusta para la carga en memoria, navegación y manipulación completa (operaciones CRUD) de estructuras jerárquicas de datos de manera controlada.
- **Persistencia y Serialización:** Se ha logrado garantizar la persistencia de la información mediante la transformación y volcado del árbol de nodos DOM a un

archivo físico en el sistema ([productos.xml](#)), asegurando que los datos se mantengan intactos entre distintas ejecuciones del aplicativo.

- **Relevancia Empresarial del XML:** El análisis teórico confirma que, pese al auge de formatos más ligeros y ágiles como JSON, el formato XML mantiene su hegemonía indiscutible en entornos empresariales críticos que requieren una validación estructural estricta, estandarización legal y alta seguridad, tales como la facturación electrónica (FacturaE) y los servicios web corporativos (SOAP).
- **Gestión Lógica de la Información:** La implementación del sistema de colecciones evidencia la necesidad técnica de organizar los documentos masivos en estructuras de directorios jerárquicas. Escalado a un entorno corporativo real, este enfoque hace indispensable la aplicación de políticas de retención de datos y estrategias de almacenamiento escalonado (Hot y Cold Storage) para optimizar el rendimiento y los costes.